Image Data Augmentation and Image Classification

Name: Saral Gupta

Course: B.Tech in Electrical Engineering

Batch: 2023-2027

Institute Name: Indian Institute of Engineering Science and Technology, Shibpur

Period of Internship: 18th May, 2026 – 14th June, 2026

Report submitted to:

IDEAS – Institute of Data Engineering, Analytics and Science Foundation, ISI Kolkata

Abstract

This internship project focuses on image data augmentation, image preprocessing, dataset loading, and machine learning evaluation using Python.

The first part of the project uses OpenCV to perform basic image processing operations. These include loading an image, converting it to grayscale, saving the processed grayscale image, shifting the image, resizing it, and rotating it. These operations demonstrate how digital images can be represented, modified, and prepared for further analysis.

The second part of the project builds an image augmentation and data loading pipeline using PyTorch and Torchvision. A cat/dog image dataset is loaded using torchvision.datasets.ImageFolder, and a transformation pipeline is applied to resize images to 255 x 255, randomly flip them horizontally, and convert them into tensors. The transformed images are then loaded in mini-batches using PyTorch DataLoader.

The final part of the project applies K-Means clustering on the sklearn digits dataset. Since K-Means produces arbitrary cluster labels, eachcluster is mapped to the most frequent true digit label before evaluation. The model performance is measured using macro-averaged F1 score, achieving a score of 0.8627854786352394.

Overall, the project demonstrates the foundational steps involved in a computer vision pipeline, including preprocessing, augmentation, dataset preparation, and basic machine learning evaluation.

Introduction

Project Overview

Image processing and machine learning are important areas in modern data science, especially in applications involving computer vision. Tasks such as object detection, image classification, medical image analysis, facial recognition, and autonomous systems rely heavily on the ability to process images and train models using visual data.

One of the key challenges in computer vision is preparing image data in a form that can be used effectively by machine learning models. Raw images often vary in size, orientation, lighting, and quality. Therefore, preprocessing and augmentation techniques are used to standardize images and increase the diversity of the training data. Image augmentation helps improve model generalization by applying transformations such as flipping, rotation, resizing, and shifting.

This project focuses on understanding the basic workflow of an image processing and image augmentation pipeline. OpenCV was used for image manipulation tasks such as grayscale conversion, affine transformation, resizing, and rotation. PyTorch and Torchvision were used to create an image transformation pipeline and load a structured cat/dog image dataset using ImageFolder.

In addition to the image processing pipeline, the project also includes a machine learning evaluation component using the sklearn digits dataset. K-Means clustering was applied to group digit images into 10 clusters. Since K-Means is an unsupervised algorithm, its cluster labels were mapped to the most frequent true labels before calculating the macro F1 score.

The project is relevant because it demonstrates the essential stages required before building a complete image classification model. These stages include preprocessing, augmentation, dataset organization, batch loading, and evaluation.

The workflow developed in this project can be extended in the future to train a supervised convolutional neural network for cat/dog classification.

Topics Covered

During the internship, the following topics were covered:

- Python programming for data science
- NumPy for numerical operations
- OpenCV for image processing
- Image transformations such as grayscale conversion, shifting, resizing, and rotation
- PyTorch and Torchvision basics
- Image augmentation using Torchvision transforms
- Dataset loading using ImageFolder
- Mini-batch creation using DataLoader
- Scikit-learn machine learning workflow
- K-Means clustering
- Model evaluation using macro F1 score

Project Objective

The objective of this project was to understand and implement the fundamental steps involved in image preprocessing, image augmentation, dataset loading, and basic machine learning evaluation. The project was designed to demonstrate how raw image data can be transformed into a structured format suitable for machine learning workflows.

The main objectives of the project were:

- To load and inspect image data using OpenCV.
- To apply basic image preprocessing techniques such as grayscale conversion, image shifting, resizing, and rotation.
- To save processed image outputs, such as the generated grayscale image graymoon.jpg.
- To create an image augmentation pipeline using Torchvision transforms.
- To load a structured cat/dog image dataset using torchvision.datasets.ImageFolder.
- To generate shuffled mini-batches using PyTorch DataLoader.
- To apply K-Means clustering on the sklearn digits dataset and evaluate the result using macro-averaged F1 score.

This project does not involve hypothesis testing or sample survey-based analysis. The work is based on image processing, data preparation, and machine learning implementation using existing image and digit datasets.

The project mainly illustrates how preprocessing and augmentation form the foundation of a computer vision system. These steps are necessary before building a complete supervised image classification model.

Methodology

This section describes the complete workflow followed in the project. The methodology includes image loading, preprocessing, augmentation, dataset preparation, mini-batch creation, and machine learning evaluation.

The work can be divided into the following stages:

1. Image loading using OpenCV
2. Image preprocessing and transformation
3. Image augmentation using Torchvision
4. Cat/dog dataset loading using ImageFolder
5. Mini-batch creation using DataLoader
6. K-Means clustering on the sklearn digits dataset
7. Evaluation using macro-averaged F1 score

4.1 Tools and Libraries used

The project was implemented using Python and several widely used libraries for computer vision, data processing, machine learning, and deep learning. Each library contributed to a specific part of the workflow, from image manipulation to dataset loading and clustering evaluation.

Tools/Library	Purpose in the Project
Python	Main programming language used for the implementation.
Numpy	Used for numerical operations and creating affine transformation matrices.
OpenCV	Used for image loading, grayscale conversion, image shifting, resizing, rotation, and saving image outputs.
PyTorch	Used for tensor-based image handling and mini-batch processing.
Torchvision	Used for defining image transforms and loading the cat/dog dataset using ImageFolder
Scikit-learn	Used for loading the digits dataset and applying K-Means clustering
SciPy	Used for finding the most frequent true label in each K-Means cluster.
Jupyter Notebook	Used as the development and execution environment.

4.2 Image Loading

The first technical step in the project was to load an image using OpenCV. The image used for this task was stored in the project directory as:
internship-data/moon-pexels-frank-cone.jpg

OpenCV's cv2.imread() function was used to read the image from the local file path.

```
import numpy as np
import cv2

original_image = cv2.imread("moon-pexels-frank-cone.jpg")
print(original_image.shape)
```

```
(800, 640, 3)
```

This output represents the shape of the loaded image. The first value, 800, represents the image height in pixels. The second value, 640, represents the image width in pixels. The third value, 3, indicates that the image has three color channels.

In OpenCV, images are loaded in BGR channel order by default instead of RGB. This is important to mention because color interpretation may differ when displaying the image using other libraries such as Matplotlib.

This step confirms that the image was loaded successfully and was available for further preprocessing operations.

4.3 Grayscale Conversion

After loading the original image, the next step was to convert it into grayscale. Grayscale conversion reduces a color image from three channels to a single intensity channel. This is useful in many image processing tasks where color information is not required, or where reducing computational complexity is beneficial.

The conversion was performed using OpenCV's cv2.cvtColor() function.

```
grayscale_image = cv2.cvtColor(original_image, cv2.COLOR_BGR2GRAY)
print(grayscale_image.shape)
```

```
(800, 640)
```

The output shape confirms that the image was converted from a three-channel color image to a single-channel grayscale image. The height and width remained the same, while the channel dimension was removed.

The grayscale image was then saved to the local project directory using cv2.imwrite().

```
cv2.imwrite("graymoon.jpg", grayscale_image)
```

```
True
```

The generated output file was: graymoon.jpg

This output file serves as evidence that the grayscale preprocessing step was completed successfully.

4.4 Image Shifting

Image shifting, also known as image translation, moves image content along the horizontal and vertical axes. In this project, the image was shifted 50 pixels to the right (t_x=50) and 100 pixels downward (t_y=100).

To execute this, an affine transformation matrix M was constructed using NumPy:

$$M = \begin{bmatrix} 1 & 0 & 50 \\ 0 & 1 & 100 \end{bmatrix}$$

This matrix keeps the image scale unchanged while shifting the content by the specified number of pixels. As shown in Figure 1, the original height and width were extracted, and the transformation was applied using OpenCV's cv2.warpAffine() function.

```
t_x, t_y = 50, 100
(h, w) = original_image.shape[:2]
M = np.float32([[1, 0, t_x], [0, 1, t_y]])

shifted_image = cv2.warpAffine(original_image, M, (w, h))

print(shifted_image.shape)
```

```
(800, 640, 3)
```

The execution output confirms that the final image shape (800, 640, 3) remains identical to the original. Because the canvas size was kept constant, only the visible content was translated within the existing frame. This technique is

frequently used in image augmentation to improve model robustness against positional variations.

4.5 Image Resizing

Image resizing was performed to alter the spatial dimensions of the original image. This preprocessing step is crucial in computer vision because deep learning models generally require input images to have a fixed size.

In this project, the target dimensions were set to a width of 150 pixels and a height of 100 pixels. The operation was executed using OpenCV's cv2.resize() function, as demonstrated in Figure 2.

```
resize_w = 150
resize_h = 100
resized_image = cv2.resize(original_image, (resize_w, resize_h))

print(resized_image.shape)
```

```
(100, 150, 3)
```

The execution output follows OpenCV's standard image format convention of (height, width, channels). This confirms that the resized image has a height of 100 pixels, a width of 150 pixels, and 3 color channels. Resizing standardizes inputs and reduces computational costs when processing large datasets.

4.6 Image Rotation

Image rotation was performed to rotate the original image 90 degrees counter-clockwise around its center. This transformation is a standard image augmentation technique used to improve model robustness against changes in object orientation.

To execute this, an affine transformation matrix was generated using OpenCV's cv2.getRotationMatrix2D() function, taking three parameters:

1. **Center:** The calculated center point around which the image rotates. It is calculated as (w // 2, h // 2).
2. **Angle:** The angle of rotation in degrees (90).
3. **Scale:** A scale factor of 1.0, ensuring the original image scale is preserved.

As shown in Figure 3, the rotation was then applied using cv2.warpAffine().

```
center = (w // 2, h // 2)

rotation_angle = 90
matrix = cv2.getRotationMatrix2D(center, rotation_angle, 1.0)

rotated_image = cv2.warpAffine(original_image, matrix, (h, w))

print(rotated_image.shape)
```

```
(640, 800, 3)
```

The execution output shows that the height and width were swapped after rotation, changing the output shape from (800, 640, 3) to (640, 800, 3).

4.7 Image Augmentation Pipeline

After exploring basic OpenCV transformations, an image augmentation pipeline was established using PyTorch and Torchvision. Augmentation increases data diversity, helping prepare datasets for machine learning workflows.

The pipeline was created using transforms.Compose(), as shown in Figure 4.

```
import torch
from torchvision import transforms


train_transform = transforms.Compose([transforms.Resize((255, 255)),
                                      transforms.RandomHorizontalFlip(), transforms.ToTensor()])
```

The pipeline included three sequential transformations:

1. **Resize((255, 255)):** Resizes every image to a fixed 255x255 dimension, ensuring uniform input tensors for the neural network.

2. **RandomHorizontalFlip():** Introduces basic geometric variation. For subjects like cats and dogs, horizontal flipping does not alter the class label, making it an ideal augmentation technique.

3. **ToTensor():** Converts the image pixel values into PyTorch tensors, standardizing the format for model training workflows.

4.8 Dataset Loading using Image Loader.

After defining the augmentation pipeline, the dataset was loaded using torchvision.datasets.ImageFolder. This method is highly efficient when images are organized into separate directories according to their class labels.

As shown in Figure 5, the dataset relies on a specific directory structure with separate train and test folders, each containing subdirectories for "cat" and "dog". ImageFolder automatically assigns class labels based on these folder names.

```
from torchvision import datasets

train_dataset = datasets.ImageFolder("Cat_Dog_data/train", transform=train_transform)


print(len(train_dataset))
```

The execution output confirms that 22,500 training images were successfully loaded. The dataset features a perfectly balanced distribution, with 11,250 training images and 1,250 testing images allocated for each class. This balance is crucial for reducing class bias during supervised model training.

4.9 Mini Batch creation using Data Loader

To prepare the data for training, a PyTorch DataLoader was instantiated. Deep learning models process data in fixed-size mini-batches rather than loading entire dataset into memory simultaneously.

```
from torch.utils.data import DataLoader

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)

image, labels = next(iter(train_loader))
print(image.shape, labels.shape)
```

The output confirms the successful creation of image and label batches. The resulting image tensor shape [64, 3, 255, 255] indicates:

64: The number of images per batch.
3: The number of color channels (RGB).
255, 255: The standardized spatial dimensions of each image.

The parameter shuffle=True was used to ensure the training images are randomly shuffled before batching. This prevents the model from learning the dataset's underlying order and improves overall training generalization.

4.10 Loading the digit Dataset.

After finalizing the cat/dog image pipeline, the project transitioned to the sklearn digits dataset to demonstrate an unsupervised machine learning workflow. This dataset consists of small grayscale images of handwritten digits from 0 to 9.

```
from sklearn.datasets import load_digits

digits = load_digits()
X = digits.data
y = digits.target
print(X.shape, y.shape)
```

The execution output confirms the dataset contains 1,797 samples. The feature matrix X has a shape of (1797, 64), meaning each sample is a 64-dimensional vector representing an 8x8 pixel grayscale image that has been flattened. The target array y contains the 1,797 true digit labels corresponding to each image.

4.11 K-Means Clustering

K-Means clustering was applied to the digits dataset. As an unsupervised algorithm, K-Means groups data points into clusters based entirely on feature similarity, without access to target labels. Because the dataset represents 10 distinct digit classes, the n_clusters parameter was set to 10, as demonstrated in Figure 8.

```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=10, random_state=42)

kmeans.fit(X)

kmeans_labels = kmeans.labels_
```

The parameter random_state=42 was included to ensure reproducibility across repeated runs.

It is important to note that K-Means does not inherently know the actual digit labels. It only identifies geometric patterns in the flattened pixel data. Therefore, the cluster indices generated by the model (e.g., Cluster 0) do not directly correspond to the real digit classes (e.g., Digit 0). An additional cluster-label mapping step is required before model performance can be accurately evaluated.

4.12 Cluster-Label Mapping and F1 Score Evaluation

Since K-Means is an unsupervised algorithm, the cluster labels it generates are arbitrary and do not automatically correspond to the actual digit labels. For example, "Cluster 3" might primarily contain images of the digit 8.

To evaluate clustering performance meaningfully, each cluster was mapped to its most frequent true digit label using the mode() function from scipy.stats. After establishing this cluster-to-label mapping, the K-Means labels were converted into predicted digit labels, and a macro-averaged F1 score was calculated. The implementation and results are shown in Figure 10.

```
from sklearn.metrics import f1_score
from scipy.stats import mode

label_map = {}

for cluster in range(10):
    true_labels_in_cluster = y[kmeans_labels == cluster]
    most_common_label = mode(true_labels_in_cluster, keepdims=True).mode[0]
    label_map[cluster] = most_common_label

mapped_labels = [label_map[label] for label in kmeans_labels]

f1 = f1_score(y, mapped_labels, average="macro")
print(f1)
```

The execution yielded a score of approximately 0.863. The macro F1 score was chosen because it gives equal importance to all digit classes, preventing high performance on only a few classes from skewing the results in a multi-class evaluation. This final score indicates that the K-Means clusters aligned reasonably well with the actual digit classes.

Evaluation Process Summary:

Cluster generation: K-Means assigns each digit sample to one of 10 clusters.

Label mapping: Each cluster is mapped to its most frequent true digit label.

Prediction conversion: Cluster labels are converted into mapped digit labels.

Metric calculation: Macro F1 score is calculated using true and mapped labels.

Final Results:

Evaluation Metric	Macro F1 score
Average type	Macro
Final Score	0.8627854786352394

Data Analysis and Results

This section presents the outputs and results obtained from the project. The analysis is divided into three main parts: image processing results, cat/dog dataset preparation results, and K-Means clustering evaluation results.

5.1 Image Processing Results

The OpenCV-based image processing tasks were successfully completed. The original image was loaded, converted to grayscale, saved as an output file, shifted, resized, and rotated.

Operation	Input Shape	Output Shape
Load Image	N/A	(800, 640, 3)
Grayscale Conversion	(800, 640, 3)	(800, 640)
Save Grayscale Image	(800, 640)	graymoon.jpg
Shift Image	(800, 640, 3)	(800, 640, 3)
Resize Image	(800, 640, 3)	(100, 150, 3)
Rotate Image	(800, 640, 3)	(640, 800, 3)

These results show that the image preprocessing operations were performed correctly. The output shapes match the expected behavior of each transformation.

5.2 Cat/Dog Dataset Summary

The cat/dog dataset was successfully loaded using Torchvision ImageFolder. The dataset was organized into class-wise folders, which allowed automatic class label assignment.

Dataset Split	Cat Images	Dog Images	Total Images
Train	11,250	11,250	22,500
Test	1,250	1,250	2500

The dataset is balanced, with an equal number of cat and dog images in both the training and testing folders. This is useful because balanced datasets reduce the risk of class bias in supervised classification tasks.

5.3 DataLoader Output

The transformed cat/dog images were successfully loaded into mini-batches using PyTorch DataLoader.

Component	Result
Training images found	22,500
Batch Size	64
Image Batch Shape	torch.Size([64, 3, 255, 255])
Label batch shape	torch.Size([64])
Shuffled Enabled	True

5.4 K-Means Clustering Results

The sklearn digits dataset was used to evaluate unsupervised clustering performance. The feature matrix contained 1,797 samples, each represented by 64 features.

Component	Value
Dataset	sklearn digits
Number of samples	1797
Number of features	64
Number of clusters	10
Algorithm	K-Means Clustering
Random State	42

After K-Means clustering, each cluster was mapped to the most frequent true digit label in that cluster. The mapped labels were then evaluated using macro-averaged F1 score.

Metric	Value
Evaluation metric	Macro F1 score
Average type	Macro
Final Score	0.8627854786352394

The achieved macro F1 score indicates that the clusters were reasonably aligned with the true digit classes after cluster-label mapping.

Conclusion

This project successfully demonstrated the foundational workflow of image preprocessing, image augmentation, dataset loading, and machine learning evaluation. The implementation was carried out using Python in a Jupyter Notebook environment.

In the first part of the project, OpenCV was used to perform basic image processing operations on the source image internship-data/moon-pexels-frank-cone.jpg. The image was successfully loaded with shape (800, 640, 3),
to grayscale with shape (800, 640), and saved as graymoon.jpg. Additional transformations such as image shifting, resizing, and 90-degree rotation were also performed successfully.

In the second part of the project, a Torchvision image augmentation pipeline was created for the cat/dog dataset. The pipeline resized images to 255 x 255, randomly flipped images horizontally, and converted them into PyTorch tensors. The dataset was loaded using torchvision.datasets.ImageFolder, and the training set contained 22,500 images. A PyTorch DataLoader was then used to generate mini-batches of size 64, with image tensors of shape torch.Size([64, 3, 255, 255]).

In the final part of the project, K-Means clustering was applied to the sklearn digits dataset. The dataset contained 1,797 samples with 64 features each. Since K-Means is an unsupervised algorithm, the generated cluster labels were mapped to the most frequent true digit labels before evaluation. The final macro-averaged F1 score was 0.8627854786352394, indicating that the clustering results aligned reasonably well with the actual digit classes.

Overall, the project helped build a clear understanding of the steps involved in preparing image data for machine learning workflows. It also demonstrated how traditional image processing, deep learning data preparation, and unsupervised learning evaluation can be combined in a single project.

For future work, the cat/dog dataset pipeline can be extended by implementing a supervised image classification model such as a convolutional neural network or a transfer learning model. Additional augmentations such as random rotation, color jitter, random cropping, and normalization can also be added to improve model robustness.

Appendices

Appendix A: References

1. OpenCV Documentation
   https://docs.opencv.org/
2. PyTorch Documentation
   https://pytorch.org/docs/stable/index.html
3. Torchvision Documentation
   https://pytorch.org/vision/stable/index.html
4. Torchvision Transforms Documentation
   https://pytorch.org/vision/stable/transforms.html
5. Scikit-learn Documentation
   https://scikit-learn.org/stable/
6. Scikit-learn KMeans Documentation
   https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html
7. Scikit-learn Digits Dataset Documentation

https://scikitlearn.org/stable/modules/generated/sklearn.datasets.load_digits.html

8. SciPy Documentation
   https://docs.scipy.org/doc/scipy/

9. Cat/Dog Dataset Source
   https://s3.amazonaws.com/content.udacity-data.com/nd089/Cat_Dog_data.zip

10. Source image file used in the project

internship-data/moon-pexels-frank-cone.jpg

Appendix B: Code Repository

GitHub Repository Link:
https://github.com/saral-gupta7/Internship-IDEAS-TIH-project.git

Main project notebook:
09_Imagedata_Augmentation_and_Image_Classification_Spring_2026.ipynb

Important project files:

- Cat_Dog_data/
- internship-data/moon-pexels-frank-cone.jpg
- graymoon.jpg
- requirements.txt
- README.md